Assignment #4: Guessing Game

Goal

In this assignment, you will deepen your understanding of network protocols by implementing a program in C or C++ of a guessing game where the server picks a random number between 1 and 100 and clients try to guess the number. The server will reply "higher" or "lower" to each guess. When a client guesses correctly, a message is sent to all connected clients telling them which client won, disconnects all clients, and resets the game.

Technology

This project is designed for Visual Studio 2022 or Visual Studio 2026. Your submission will be tested on a DigiPen lab-machine equivalent: Windows 10 or 11 and latest update of Visual Studio 2022 / 2026.

Example code can be downloaded from
`https://github.com/DigiPen-Network-Classes/guessing-game-student`

Or using git:

```
git clone https://github.com/DigiPen-Network-Classes/guessing-game-student.git
```

The Game

You will be writing both a Client and a Server for the guessing game.

The server will pick a number at random between 1 and 100. The server also needs to keep track of if the game has been won, and which client was the last winner.

Clients will connect simultaneously to the server. The server should assign an ID to each client – a simple atomic integer that is incremented one by one is good enough.

The server will send each client a welcome message and ask for a guess. Each client will send their guess, and clients will receive a message in response indicating if their guess was Too Low, Too High, or Correct.

When a client guesses correctly, all clients are told the ID of the winner, and then all clients are (safely and correctly) shutdown. Then the game server should reset itself – picking a new random number and getting ready for new connections.

Dumb Client / Server Model

Your solution should consist of two projects: the server and the client. The client is a "dumb client" – its only job is to connect to the server and read input from the user. The output telling the user what to do should come from the server.

Programming

Both the server and client will take one argument from the command line – the port that the server will listen to and the client will connect to. The server should listen to the loopback interface on 127.0.0.1. The client will then connect to the server using TCP.

If the port argument is not given, or is invalid (not in the range 1-65535, not an integer, etc.) then your program should print out a simple usage statement.

```
std::cerr << "Usage: " << argv[0] << " <port>\n";
```

Example Usage and Output (Server)

```
> GuessingGameServer.exe 2222
Server listening on port 2222
client 1 connected
client 1 disconnected
```

Example Usage and Output (Client)

```
> GuessingGameClient.exe 2222
Welcome to the guessing game!
Guess a number between 1 and 100.
Your guess: 50
Too high.
Your guess:
Client 2 has won! Disconnecting ...

>
```

Tasks

Game Server Tasks

Multi-tasking Server

The game server must handle many clients simultaneously – you will need to consider how you want to implement this. The server will bind() to the correct port and address (127.0.0.1), then listen() and accept() clients. Any "game setup" (picking the number to guess) must happen before the first accept() call.

One approach is to have the main thread call accept(), and then hand all received client sockets off to their own threads. This simplifies the socket api work, but means you'll have to use mutexes and lock guards to keep the threads from conflicting with each other.

Thread Example

```
#include <thread>
while(true)
{
  // if game is over, reset accordingly ...
  SOCKET client_socket = accept(server_socket, nullptr, nullptr);
  // the detach() is important!
  std::thread(handle_client, client_socket).detach();

  // then sleep the main thread to avoid problems
  std::this_thread::sleep_for(std::chrono::milliseconds(100));

  // other code ..
}
```

Alternatively, you could use select() and traverse a list of sockets, polling each one to get guesses. This can be done in a single thread, but it requires a little more code to implement correctly – you might find it easier to use select and skip the threading issues: managing the state of each socket, who won, is the game ongoing, etc.

Tracking Client IDs

The server will assign each client an ID when they connect – an integer is fine. You could use the SOCKET handle itself, or just increment an atomic integer.

```
void handle_client(SOCKET sock)
{
  static std::atomic<int> client_counter { 0 };
  int client_id = ++client_counter;
  // etc.
}
```

The client should be told what its ID is as part of your protocol and setup – perhaps with the initial "welcome" message from the server to the client.

Winner!

When a client guesses the number, the server must send a message to all clients telling them which client won, and then shutdown() all the connections cleanly. The clients are disconnected and must terminate when the game is over.

After, the server must reset its game state (starting a new guessing game) and wait for new clients to connect. The server must put itself back in a state where it can accept more connections play the game again with guesses vs. the new number. The server must NOT

terminate. However, while you're debugging you may find it easier to implement a version where the server process exits at the end of the game and then correct the server behavior once you've got the implementation working.

Example Server Game Code

This is provided as an example, you don't have to use this code in your solution.

Note the use of lock_guard and a mutex – if you use multiple threads, you must make sure that two threads do not attempt to alter the same data at the same time! Saying "it works on my machine" is not sufficient!

Failure to safely handle multiple threads, in both the server and the client, -- if you choose to use multiple threads -- will result in point deduction!

```
void init_game()
{
  std::srand(static_cast<unsigned int>(std::time(nullptr)));
  std::lock_guard<std::mutex> lock(game_mutex);
  secret_number = std::rand() % 100 + 1;
  winner_client = -1;
  game_over_flag = false;
  // etc..
}
```

Client Tasks

The client takes as an argument the port number it should attempt to connect to. Print an error including usage if the argument is not given, or is not valid (outside of the valid range 1-65535, not a number, and so on).

The client connects to the server at 127.0.0.1 using that port. While the client's connection exists, recv() the prompts from the server and read input from the user.
NOTE: you can output the "Your Guess?" prompt from the client. See the client example for more information.

When the game is over all clients must receive a message that there was a winner, what the winning guess was, and then be shut down.

Reading Input

The code to read input must be done in a memory-safe way! Using gets() is a compile-time error!

The example client creates a thread to process input and uses the main thread to handle the recv loop. You do not have to use that implementation, but your implementation must be thread-safe.

Suggestions

The client program may either use blocking or non-blocking sockets. The server will need to use some mechanism for managing multiple simultaneous clients.
You may define a simple protocol for the client and server to use: who sends first, who responds? How are guesses sent – as strings, as an integer, something else?
Your protocol may include a text message and some indicator that the game is over and who has won. You may want to add some final end-of-game message between client and server so that the client acknowledges the game is over. In any case, clients should disconnect cleanly -- shutting down, not just disconnecting or terminating.

Beware making this protocol too complicated! Keep it simple!

Testing

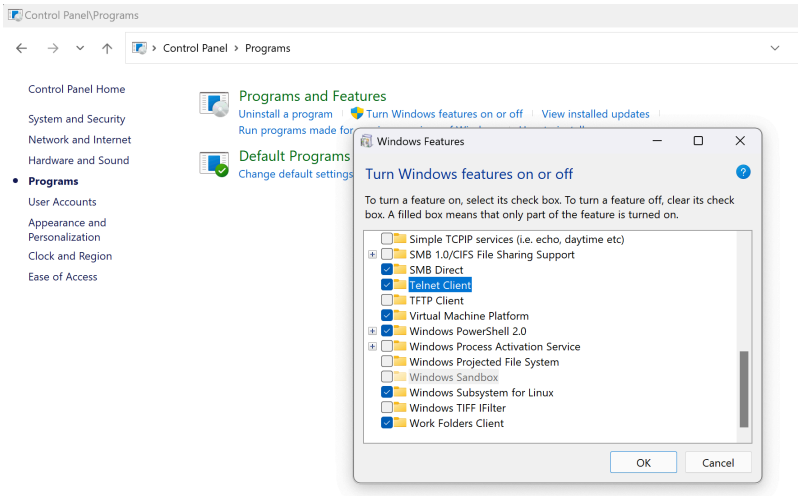
Initially, you can use the telnet client to test your work. This is an incredibly simple tool for sending and receiving data – however it sends and receives everything "in the clear" so you should never use it for anything important!

Note: *You only need to use telnet as a tool for testing your server*, before you've got a client written – this helps get around the Chicken and Egg problem of writing a server without a client, or writing a client without a server…

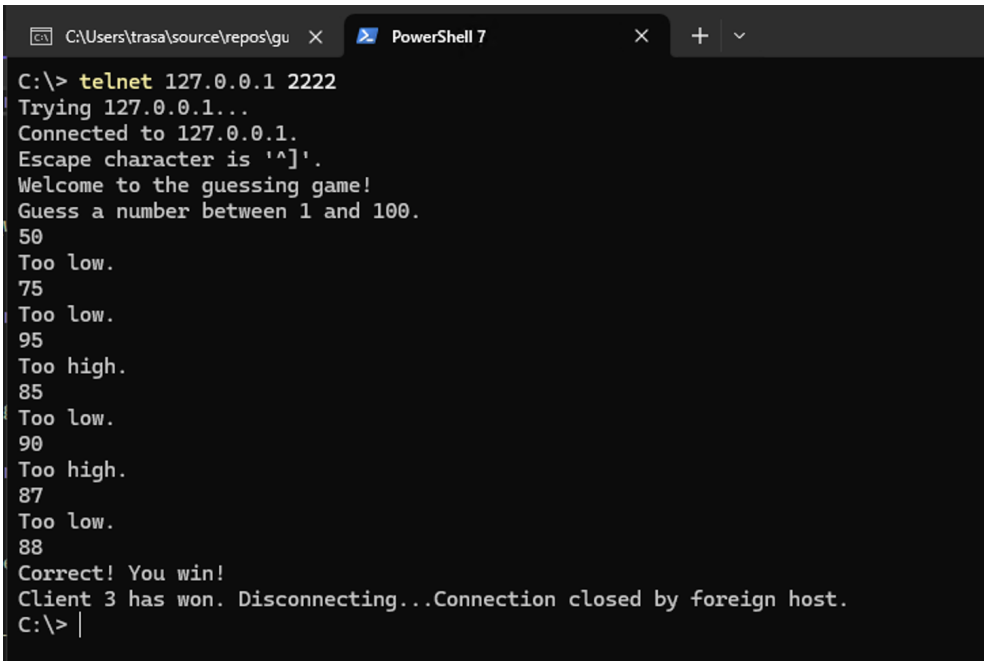
Your server does not need to work correctly with telnet clients, and your server won't be tested using telnet. The client and server will be tested together.

Enabling Telnet Client (Windows)

Open "Control Panel" and select "Enable or Disable Windows Features". In that list of features, find "Telnet Client", click OK, and follow instructions to finish the installation.

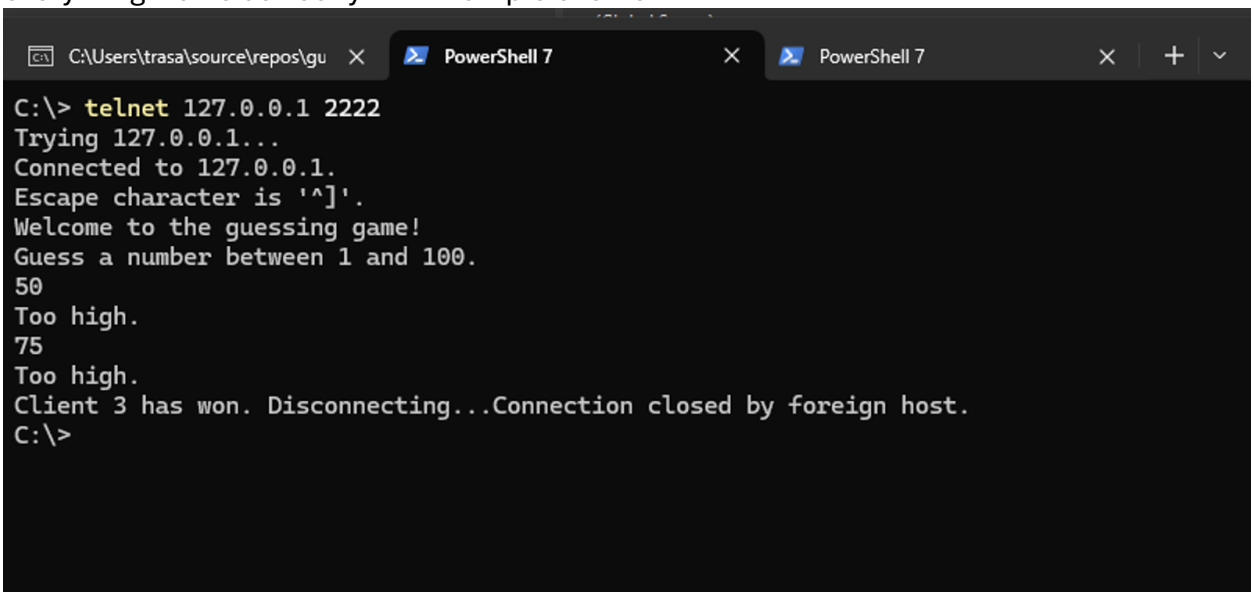


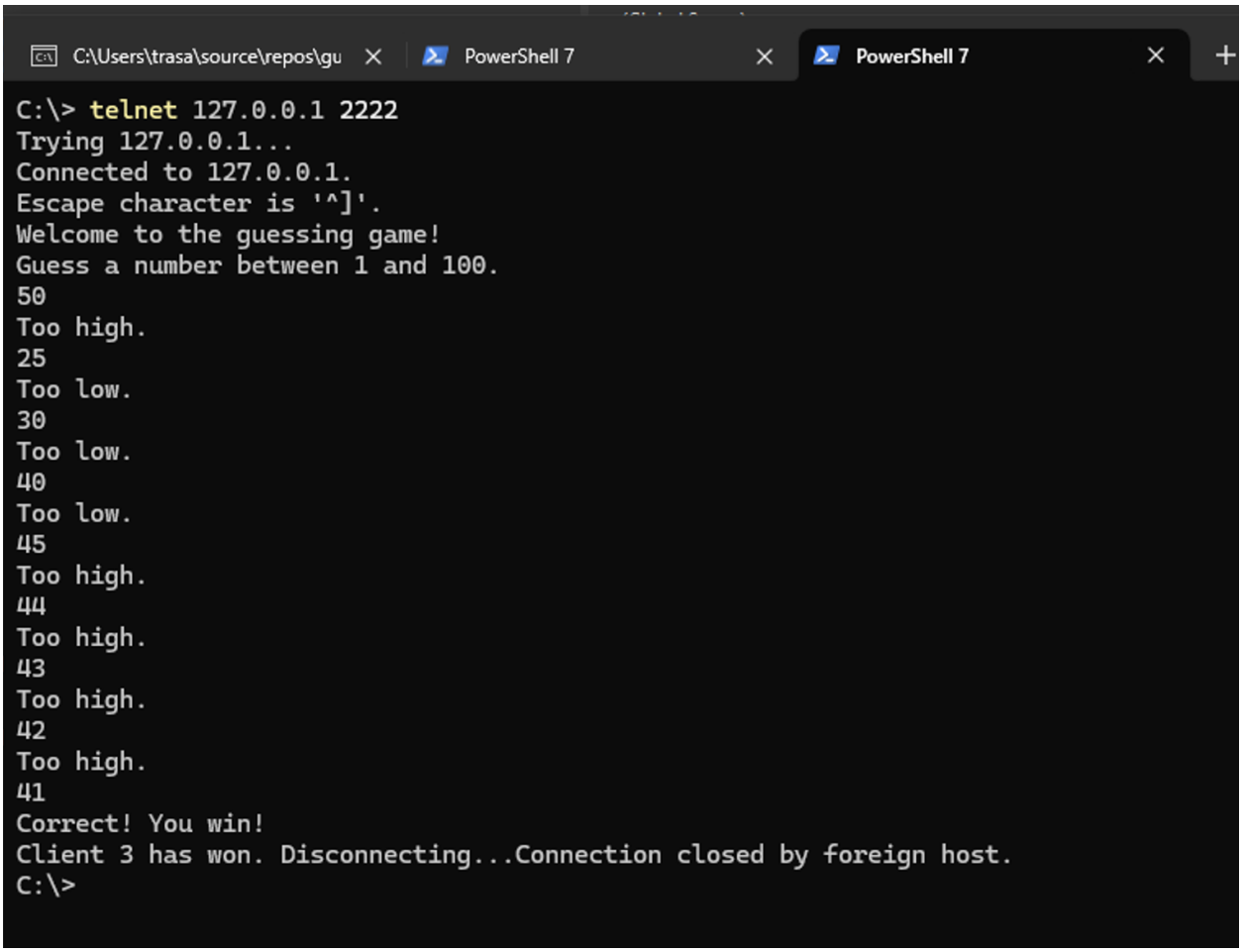
Once you have telnet installed, you can test your server via terminal window:



Testing with Multiple Clients

You can use the Windows Terminal program to run multiple copies of telnet, testing that everything works correctly with multiple clients:





Submission

Your project should be named GuessingGame.exe and GuessingGameClient.exe. You should not do anything fancy with paths or copying binaries or using cmake, etc. It should be buildable “as is” using Visual Studio 2022.

You should zip up the solution, project, and source files – but DO NOT include extra directories such as an extra top level directory, and DO NOT include build product (debug/release builds and folders).

You may include a README that contains any comments or questions – anything you think I should know about. This is optional.

Your client and server will be tested together – other clients such as telnet will not be used to test the server.

Submission File Name

You must name the archive YourAlias-CS260-GuessingGame.zip where “YourAlias” is the username used for Moodle.

Grading

From a base score of 100, you’ll lose points for the following errors:

- -100 if your solution cannot be compiled
    - Do not rely on external tools like cmake!
- -100 if your client uses gets() instead of a safe input method. gets() is considered a compile-time error.
- -70 if your programs do not send and receive messages correctly.
- -30 if your server does not support multiple connections correctly.
- Between -10 and -40 (depending on severity) if your program does not implement multithreading in a safe manner, if threads are used.
- -30 if your server program does not reset correctly after a game is over.
- -10 if your client or server does not parse command arguments correctly, or allows invalid input.
- -10 if your code compiles with any warnings.
- (Various) Other penalties will be assessed depending on the accuracy of the output, handling error conditions correctly, and other programming mistakes.

Code Quality

Your program should look “finished” and presentable – no large blocks of commented code; no excessive “DEBUG PRINT” output. Penalties will be assessed based on the severity of the presentation.

Your code must compile on all platforms and configurations included with the Solution, with no warnings at the default warning level of 3. Warnings will incur a 10-point penalty. Do not change the warning level for the assignment (unless otherwise specified) -- this includes Code Analysis warnings as well as build output, but not tasks or hints)

Submission Penalties

- (-10) if your submission is named incorrectly
- (-10) if your output is named incorrectly
- (-10) if your submission includes additional files or directories
- (-10) if your submission includes binaries